

Self-Replication SOP: Spawning a New Autopilot Instance

Status: Ready — all pieces exist today

Trigger: Governor says "Sophia, spawn a new instance for [name]" or "Sophia, fork the ecosystem for [name]"

1. Overview

There are **two paths** for spawning a new instance:

Path	What it creates	When to use
Clone	A new Sophia that shares the existing Edgar API and ledger	Same DAO, new operator (e.g. Bilal wants his own assistant to manage the same cacao supply chain)
Fork	A completely new ecosystem: new Sophia + new Edgar + blank Google Sheet ledger + blank context	New DAO, new rules (e.g. Liz wants to run a coffee co-op with different tokenomics)

2. Decision Tree

Do you want to manage the SAME ecosystem?

- **Yes** → **CLONE** — Same Edgar, same ledger, same QR codes. Just need a new operator interface. ~10 minutes, 3 credentials from governor.
- **No** → **FORK** — New everything. You write your own rules. ~1 hour, governor defines their own operating instructions.

3. Clone Path (Same DAO, New Operator)

Governor provides (3 credentials)

Credential	Why
LLM API Key	Required for the new instance to reason and respond
GitHub PAT	Required to read repos, open PRs, submit contributions
Telegram Bot Token	Required to receive/send messages

Sophia provides (from existing vault)

Resource	Source
AWS keys (EC2 provisioning)	Already in vault
SSH keys	Already in vault
GitHub PAT (for repo creation)	Already in vault
Codebase (truesight_autopilot)	Public GitHub repo
Context repo fork	Forked from agentic_ai_context with existing content

Steps

1. **Provision server** — EC2 instance via AWS keys in vault
2. **Clone codebase** — git clone truesight_autopilot
3. **Register DAO identity** — register_identity() for the new instance
4. **Fork context repo** — GitHub API, seeded with existing runbooks
5. **Create transcript repo** — empty GitHub repo
6. **Configure .env** — point to the forked context repo and new transcript repo
7. **Start service** — systemctl on the target host
8. **Governor provisions credentials** — visits new vault UI, adds 3 non-negotiables
9. **Verify** — /vault/status loads, bot responds

4. Fork Path (New DAO, New Rules)

Governor provides

Item	Details
3 non-negotiable credentials	LLM API key, GitHub PAT, Telegram bot token
Operating instructions	What rules should the new autopilot follow? Written as OPERATING_INSTRUCTIONS.md
Tokenomics / ledger schema	How should value flow? What's the currency?
Governor identity	Who are the governors of the new DAO?
AWS account (optional)	If they want infrastructure separate from TrueSightDAO's AWS

Sophia provisions

Component	What gets created
New Sophia	Fresh autopilot instance with its own identity

Component	What gets created
New Edgar	Fresh FastAPI instance (dao_protocol) — no PostgreSQL needed, uses Google Sheets + Apps Script + GitHub Actions
New ledger	Blank Google Sheet for the new DAO's transactions
New Google Apps Script	Deployments pointing at the new sheet
New GitHub data repos	For public caches (treasury-cache, places-cache, etc.)
New context repo	Forked from agentic_ai_context with blank starter templates only
New transcript repo	Empty GitHub repo
New vault	Encrypted with its own Fernet key
Infrastructure	EC2 instance(s), security groups, DNS (if applicable)

Steps

1. **Governor defines the rules** — writes OPERATING_INSTRUCTIONS.md for the new ecosystem
2. **Provision infrastructure** — EC2 for Sophia + Edgar
3. **Clone both repos** — truesight_autopilot + dao_protocol
4. **Set up Edgar** — configure .env with new Google Sheet ID, new service accounts, new Apps Script deployment IDs
5. **Register DAO identity** — for the new instance
6. **Create blank context repo** — starter templates only
7. **Create transcript repo** — empty
8. **Create GitHub data repos** — for public caches
9. **Start both services** — Sophia + Edgar
10. **Governor provisions credentials** — visits new vault UI
11. **Verify** — /vault/status shows both Sophia and Edgar as healthy

5. Credential Handoff Flow

Clone

Governor → "Spawn for Bilal" → Sophia provisions EC2, clones, configures, starts service → reports URL → Governor visits vault UI, adds 3 credentials, sends test message → "I'm alive!"

Fork

Governor → "Fork for Liz" + provides operating instructions + tokenomics → Sophia provisions infra, clones Sophia + Edgar, creates blank context, starts both services → reports URLs → Governor visits vault UI, adds credentials, writes operating rules

6. Post-Spawn Verification Checklist

Clone

- [] /vault/status loads and shows runtime config
- [] Vault shows the 3 non-negotiables
- [] Telegram bot responds to /start
- [] Governor can send a message and get a response
- [] New instance can read the shared Edgar API

Fork

- [] /vault/status loads on the new Sophia
- [] Edgar API responds at its endpoint
- [] New ledger sheet is accessible
- [] Governor's operating instructions are loaded
- [] Telegram bot responds
- [] Governor can send a message and get a response following the NEW rules

7. Architecture Notes

- **Edgar has no PostgreSQL** — it uses Google Sheets as the ledger, Google Apps Script for serverless functions, and GitHub Actions for scheduled jobs. A fork just needs new sheet IDs and new Apps Script deployments.
- Each instance has its **own** vault encrypted with its **own** Fernet key. Credentials are never shared between instances.
- A Fork creates a completely independent DAO — no shared infrastructure, no shared data, no shared governance.
- The governor can destroy the instance at any time by terminating the EC2 instance and deleting the repos.
- For a quick test, the governor can also run the autopilot locally with `python3 app/main.py`.